
14 Operations On Bits

- Bitwise Operators
 - One's Complement Operator
 - Right Shift Operator
 - Left Shift Operator
 - Bitwise AND Operator
 - Bitwise OR Operator
 - Bitwise XOR Operator
- The *showbits()* Function
- Summary
- Exercise

So far we have dealt with characters, integers, floats and their variations. The smallest element in memory on which we are able to operate as yet is a byte; and we operated on it by making use of the data type **char**. However, we haven't attempted to look within these data types to see how they are constructed out of individual bits, and how these bits can be manipulated. Being able to operate on a bit level, can be very important in programming, especially when a program must interact directly with the hardware. This is because, the programming languages are byte oriented, whereas hardware tends to be bit oriented. Let us now delve inside the byte and see how it is constructed and how it can be manipulated effectively. So let us take apart the byte... bit by bit.

Bitwise Operators

One of C's powerful features is a set of bit manipulation operators. These permit the programmer to access and manipulate individual bits within a piece of data. The various Bitwise Operators available in C are shown in Figure 14.1.

Operator	Meaning
~	One's complement
>>	Right shift
<<	Left shift
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR(Exclusive OR)

Figure 14.1

These operators can operate upon **ints** and **chars** but not on **floats** and **doubles**. Before moving on to the details of the operators, let

us first take a look at the bit numbering scheme in integers and characters. Bits are numbered from zero onwards, increasing from right to left as shown below:

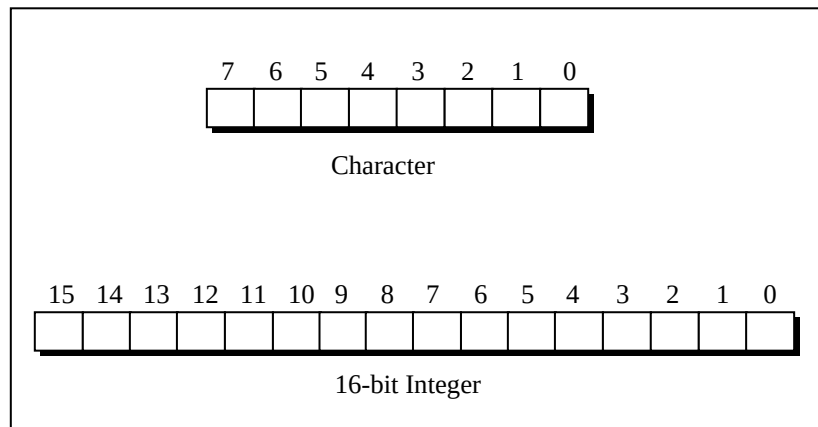


Figure 14.2

Throughout this discussion of bitwise operators we are going to use a function called **showbits()**, but we are not going to show you the details of the function immediately. The task of **showbits()** is to display the binary representation of any integer or character value.

We begin with a plain-jane example with **showbits()** in action.

```
/* Print binary equivalent of integers using showbits( ) function */
main( )
{
    int j ;

    for ( j = 0 ; j <= 5 ; j++ )
    {
        printf ( "\nDecimal %d is same as binary ", j ) ;
        showbits ( j ) ;
    }
}
```

```
}
```

And here is the output...

```
Decimal 0 is same as binary 0000000000000000
Decimal 1 is same as binary 0000000000000001
Decimal 2 is same as binary 0000000000000010
Decimal 3 is same as binary 0000000000000011
Decimal 4 is same as binary 0000000000000100
Decimal 5 is same as binary 0000000000000101
```

Let us now explore the various bitwise operators one by one.

One's Complement Operator

On taking one's complement of a number, all 1's present in the number are changed to 0's and all 0's are changed to 1's. For example one's complement of 1010 is 0101. Similarly, one's complement of 1111 is 0000. Note that here when we talk of a number we are talking of binary equivalent of the number. Thus, one's complement of 65 means one's complement of 0000 0000 0100 0001, which is binary equivalent of 65. One's complement of 65 therefore would be, 1111 1111 1011 1110. One's complement operator is represented by the symbol $\sim$. Following program shows one's complement operator in action.

```
main()
{
    int j, k;

    for (j = 0 ; j <= 3 ; j++)
    {
        printf ( "\nDecimal %d is same as binary ", j );
        showbits ( j );

        k = ~j;
        printf ( "\nOne's complement of %d is ", j );
```

```

        showbits ( k );
    }
}

```

And here is the output of the above program...

```

Decimal 0 is same as binary 0000000000000000
One's complement of 0 is 1111111111111111
Decimal 1 is same as binary 0000000000000001
One's complement of 1 is 1111111111111110
Decimal 2 is same as binary 0000000000000010
One's complement of 2 is 1111111111111101
Decimal 3 is same as binary 0000000000000011
One's complement of 3 is 1111111111111100

```

In real-world situations where could the one's complement operator be useful? Since it changes the original number beyond recognition, one potential place where it can be effectively used is in development of a file encryption utility as shown below:

```

/* File encryption utility */
#include "stdio.h"
main()
{
    encrypt( ) ;
}

encrypt( )
{
    FILE *fs, *ft ;
    char ch ;

    fs = fopen ( "SOURCE.C", "r" ) ; /* normal file */
    ft = fopen ( "TARGET.C", "w" ) ; /* encrypted file */

    if ( fs == NULL || ft == NULL )
    {

```

```
        printf ( "\nFile opening error!" );
        exit ( 1 );
    }

    while ( ( ch = getc ( fs ) ) != EOF )
        putc ( ~ch, ft );

    fclose ( fs );
    fclose ( ft );
}
```

How would you write the corresponding decrypt function? Would there be any problem in tackling the end of file marker? It may be recalled here that the end of file in text files is indicated by a character whose ASCII value is 26.

Right Shift Operator

The right shift operator is represented by `>>`. It needs two operands. It shifts each bit in its left operand to the right. The number of places the bits are shifted depends on the number following the operator (i.e. its right operand).

Thus, `ch >> 3` would shift all bits in `ch` three places to the right. Similarly, `ch >> 5` would shift all bits 5 places to the right.

For example, if the variable `ch` contains the bit pattern 11010111, then, `ch >> 1` would give 01101011 and `ch >> 2` would give 00110101.

Note that as the bits are shifted to the right, blanks are created on the left. These blanks must be filled somehow. They are always filled with zeros. The following program demonstrates the effect of right shift operator.

```
main()
{
```

```
int i = 5225, j, k ;

printf ( "\nDecimal %d is same as binary ", i ) ;
showbits ( i ) ;

for ( j = 0 ; j <= 5 ; j++ )
{
    k = i >> j ;
    printf ( "\n%d right shift %d gives ", i, j ) ;
    showbits ( k ) ;
}
```

The output of the above program would be...

```
Decimal 5225 is same as binary 0001010001101001
5225 right shift 0 gives 0001010001101001
5225 right shift 1 gives 0000101000110100
5225 right shift 2 gives 0000010100011010
5225 right shift 3 gives 0000001010001101
5225 right shift 4 gives 0000000101000110
5225 right shift 5 gives 0000000010100011
```

Note that if the operand is a multiple of 2 then shifting the operand one bit to right is same as dividing it by 2 and ignoring the remainder. Thus,

```
64 >> 1 gives 32
64 >> 2 gives 16
128 >> 2 gives 32
```

but,

```
27 >> 1 is 13
49 >> 1 is 24 .
```

A Word of Caution

In the explanation `a >> b` if `b` is negative the result is unpredictable. If `a` is negative than its left most bit (sign bit) would be 1. On some computer right shifting `a` would result in extending the sign bit. For example, if `a` contains -1, its binary representation would be 1111111111111111. Without sign extension, the operation `a >> 4` would be 0000111111111111. However, on the machine on which we executed this expression the result turns out to be 1111111111111111. Thus the sign bit 1 continues to get extended.

Left Shift Operator

This is similar to the right shift operator, the only difference being that the bits are shifted to the left, and for each bit shifted, a 0 is added to the right of the number. The following program should clarify my point.

```
main()
{
    int i = 5225, j, k ;

    printf ( "\nDecimal %d is same as ", i ) ;
    showbits ( i ) ;

    for ( j = 0 ; j <= 4 ; j++ )
    {
        k = i << j ;
        printf ( "\n%d left shift %d gives ", i, j ) ;
        showbits ( k ) ;
    }
}
```

The output of the above program would be...

Decimal 5225 is same as binary 0001010001101001

5225 left shift 0 gives 0001010001101001
 5225 left shift 1 gives 0010100011010010
 5225 left shift 2 gives 0101000110100100
 5225 left shift 3 gives 1010001101001000
 5225 left shift 4 gives 0100011010010000

Having acquainted ourselves with the left shift and right shift operators, let us now find out the practical utility of these operators.

In DOS/Windows the date on which a file is created (or modified) is stored as a 2-byte entry in the 32 byte directory entry of that file. Similarly, a 2-byte entry is made of the time of creation or modification of the file. Remember that DOS/Windows doesn't store the date (day, month, and year) of file creation as a 8 byte string, but as a codified 2 byte entry, thereby saving 6 bytes for each file entry in the directory. The bitwise distribution of year, month and date in the 2-byte entry is shown in Figure 14.3.

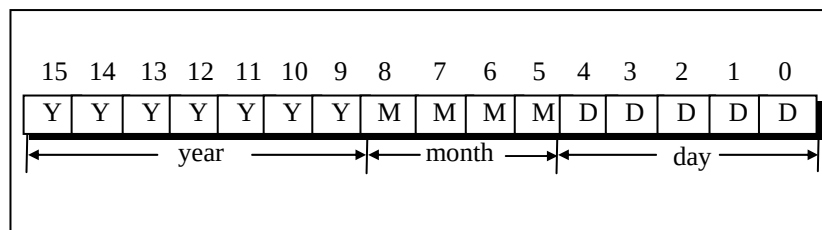


Figure 14.3

DOS/Windows converts the actual date into a 2-byte value using the following formula:

$$\text{date} = 512 * (\text{year} - 1980) + 32 * \text{month} + \text{day}$$

Suppose 09/03/1990 is the date, then on conversion the date will be,

$$\text{date} = 512 * (1990 - 1980) + 32 * 3 + 9 = 5225$$

The binary equivalent of 5225 is 0001 0100 0110 1001. This binary value is placed in the date field in the directory entry of the file as shown below.

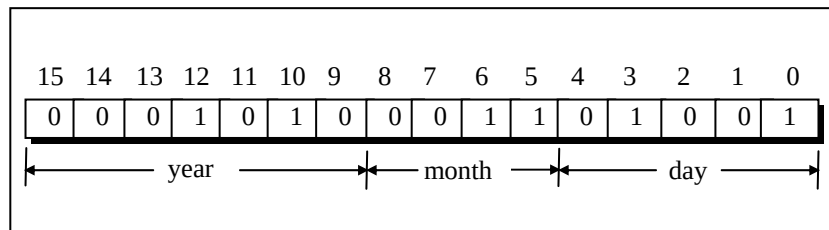


Figure 14.4

Just to verify this bit distribution, let us take the bits representing the month,

$$\begin{aligned}
 \text{month} &= 0011 \\
 &= 1 * 2 + 1 * 1 \\
 &= 3
 \end{aligned}$$

Similarly, the year and the day can also be verified.

When we issue the command DIR or use Windows Explorer to list the files, the file's date is again presented on the screen in the usual date format of mm/dd/yy. How does this integer to date conversion take place? Obviously, using left shift and right shift operators.

When we take a look at Figure 14.4 depicting the bit pattern of the 2- byte date field, we see that the year, month and day exist as a bunch of bits in contiguous locations. Separating each of them is a matter of applying the bitwise operators.

For example, to get year as a separate entity from the two bytes entry we right shift the entry by 9 to get the year. Just see, how...

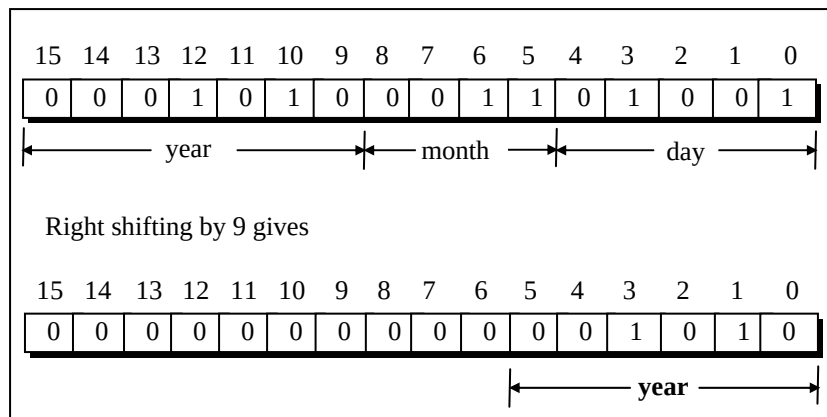


Figure 14.5

On similar lines, left shifting by 7, followed by right shifting by 12 yields month.

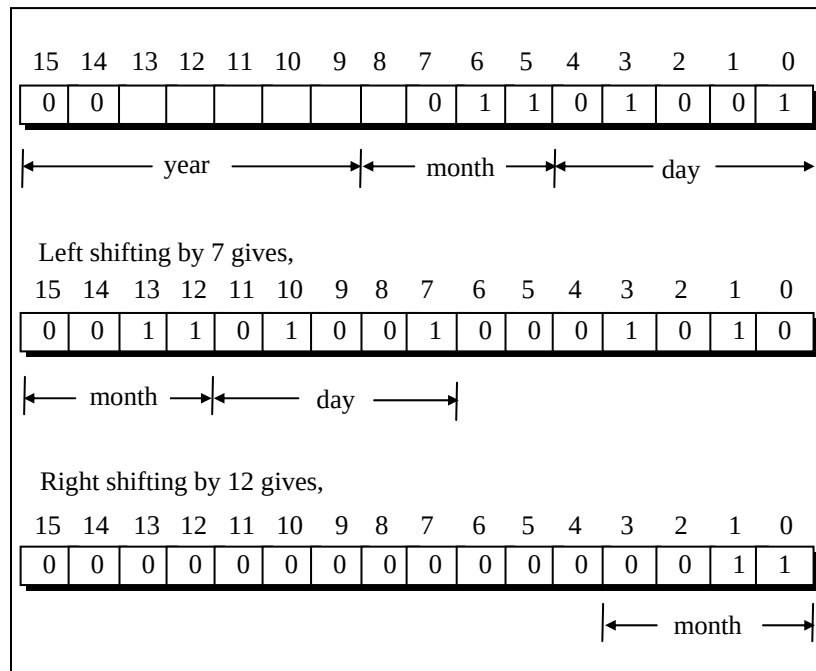


Figure 14.6

Finally, for obtaining the day, left shift date by 11 and then right shift the result by 11. Left shifting by 11 gives 0100100000000000. Right shifting by 11 gives 0000000000001001.

This entire logic can be put into a program as shown below:

```
/* Decoding date field in directory entry using bitwise operators */
main()
{
    unsigned int d = 9, m = 3, y = 1990, year, month, day, date ;

    date = ( y - 1980 ) * 512 + m * 32 + d ;
    printf ( "\nDate = %u", date ) ;
}
```

```
year = 1980 + ( date >> 9 );  
month = ( (date << 7 ) >> 12 );  
day = ( (date << 11 ) >> 11 );  
printf ( "\nYear = %u ", year );  
printf ( "Month = %u ", month );  
printf ( "Day = %u", day );  
}
```

And here is the output...

```
Date = 5225  
Year = 1990 Month = 3 Day = 9
```

Bitwise AND Operator

This operator is represented as **&**. Remember it is different than **&&**, the logical AND operator. The **&** operator operates on two operands. While operating upon these two operands they are compared on a bit-by-bit basis. Hence both the operands must be of the same type (either **char** or **int**). The second operand is often called an AND mask. The **&** operator operates on a pair of bits to yield a resultant bit. The rules that decide the value of the resultant bit are shown below:

First bit	Second bit	First bit & Second bit
0	0	0
0	1	0
1	0	0
1	1	1

Figure 14.7

This can be represented in a more understandable form as a ‘Truth Table’ shown in Figure 14.8.

&	0	1
0	0	0
1	0	1

Figure 14.8

The example given below shows more clearly what happens while ANDing one operand with another. The rules given in the Figure 14.8 are applied to each pair of bits one by one.

7	6	5	4	3	2	1	0	
1	0	1	0	1	0	1	0	This operand when ANDed bitwise
7	6	5	4	3	2	1	0	
1	1	0	0	0	0	0	1	With this operand yields
7	6	5	4	3	2	1	0	
1	0	0	0	0	0	0	1	this result

Figure 14.9

Work through the Truth Table and confirm that the result obtained is really correct.

Thus, it must be clear that the operation is being performed on individual bits, and the operation performed on one pair of bits is

completely independent of the operation performed on the other pairs.

Probably, the best use of the AND operator is to check whether a particular bit of an operand is ON or OFF. This is explained in the following example.

Suppose, from the bit pattern 10101101 of an operand, we want to check whether bit number 3 is ON (1) or OFF (0). Since we want to check the bit number 3, the second operand for the AND operation should be $1 * 2^3$, which is equal to 8. This operand can be represented bitwise as 00001000.

Then the ANDing operation would be,

10101101	Original bit pattern
00001000	AND mask

00001000	Resulting bit pattern

The resulting value we get in this case is 8, i.e the value of the second operand. The result turned out to be 8 since the third bit of the first operand was ON. Had it been OFF, the bit number 3 in the resulting bit pattern would have evaluated to 0 and the complete bit pattern would have been 00000000.

Thus, depending upon the bit number to be checked in the first operand we decide the second operand, and on ANDing these two operands the result decides whether the bit was ON or OFF. If the bit is ON (1), the resulting value turns out to be a non-zero value which is equal to the value of second operand, and if the bit is OFF (0) the result is zero as seen above. The following program puts this logic into action.

```
/* To test whether a bit in a number is ON or OFF */
main( )
{
```

```
int i = 65, j ;

printf ( "\nvalue of i = %d", i ) ;
j = i & 32 ;

if ( j == 0 )
    printf ( "\nand its fifth bit is off" ) ;
else
    printf ( "\nand its fifth bit is on" ) ;

j = i & 64 ;

if ( j == 0 )
    printf ( "\nwhereas its sixth bit is off" ) ;
else
    printf ( "\nwhereas its sixth bit is on" ) ;
}
```

And here is the output...

```
Value of i = 65
and its fifth bit is off
whereas its sixth bit is on
```

In every file entry present in the directory, there is an attribute byte. The status of a file is governed by the value of individual bits in this attribute byte. The AND operator can be used to check the status of the bits of this attribute byte. The meaning of each bit in the attribute byte is shown in Figure 14.10.

Bit numbers								Meaning
7	6	5	4	3	2	1	0	
.	.	.	.	.	.	.	1	Read only
.	.	.	.	.	.	1	.	Hidden
.	.	.	.	.	1	.	.	System
.	.	.	.	1	.	.	.	Volume label entry
.	.	.	1	.	.	.	.	Sub-directory entry
.	.	1	.	.	.	.	.	Archive bit
.	1	.	.	.	.	.	.	Unused
1	.	.	.	.	.	.	.	Unused

Figure 14.10

Now, suppose we want to check whether a file is a hidden file or not. A hidden file is one, which is never shown in the directory, even though it exists on the disk. From the above bit classification of attribute byte, we only need to check whether bit number 1 is ON or OFF.

So, our first operand in this case becomes the attribute byte of the file in question, whereas the second operand is the $1 * 2^1 = 2$, as discussed earlier. Similarly, it can be checked whether the file is a system file or not, whether the file is read-only file or not, and so on.

The second, and equally important use of the AND operator is in changing the status of the bit, or more precisely to switch OFF a particular bit.

If the first operand happens to be 00000111, then to switch OFF bit number 1, our AND mask bit pattern should be 11111101. On applying this mask, we get,

```
00000111  Original bit pattern
11111101  AND mask
-----
00000101  Resulting bit pattern
```

Here in the AND mask we keep the value of all other bits as 1 except the one which is to be switched OFF (which is purposefully kept as 0). Therefore, irrespective of whether the first bit is ON or OFF previously, it is switched OFF. At the same time the value 1 provided in all the other bits of the AND mask (second operand) keeps the bit values of the other bits in the first operand unaltered.

Let's summarize the uses of bitwise AND operator:

- (a) It is used to check whether a particular bit in a number is ON or OFF.
- (b) It is used to turn OFF a particular bit in a number.

Bitwise OR Operator

Another important bitwise operator is the OR operator which is represented as `|`. The rules that govern the value of the resulting bit obtained after ORing of two bits is shown in the truth table below.

<code> </code>	0	1
0	0	1
1	1	1

Figure 14.11

Using the Truth table confirm the result obtained on ORing the two operands as shown below.

```
11010000  Original bit pattern
00000111  OR mask
-----
11010111  Resulting bit pattern
```

Bitwise OR operator is usually used to put ON a particular bit in a number.

Let us consider the bit pattern 11000011. If we want to put ON bit number 3, then the OR mask to be used would be 00001000. Note that all the other bits in the mask are set to 0 and only the bit, which we want to set ON in the resulting value is set to 1.

Bitwise XOR Operator

The XOR operator is represented as $\wedge$ and is also called an Exclusive OR Operator. The OR operator returns 1, when any one of the two bits or both the bits are 1, whereas XOR returns 1 only if one of the two bits is 1. The truth table for the XOR operator is given below.

$\wedge$	0	1
0	0	1
1	1	0

Figure 14.12

XOR operator is used to toggle a bit ON or OFF. A number XORed with another number twice gives the original number. This is shown in the following program.

```
main()
{
    int b = 50 ;

    b = b ^ 12 ;
    printf ( "\n%d", b ) ; /* this will print 62 */

    b = b ^ 12 ;
    printf ( "\n%d", b ) ; /* this will print 50 */
}
```

The *showbits()* Function

We have used this function quite often in this chapter. Now we have sufficient knowledge of bitwise operators and hence are in a position to understand it. The function is given below followed by a brief explanation.

```
showbits ( int n )
{
    int i, k, andmask ;

    for ( i = 15 ; i >= 0 ; i-- )
    {
        andmask = 1 << i ;
        k = n & andmask ;

        k == 0 ? printf ( "0" ) : printf ( "1" ) ;
    }
}
```

All that is being done in this function is using an AND operator and a variable **andmask** we are checking the status of individual bits. If the bit is OFF we print a 0 otherwise we print a 1.

First time through the loop, the variable **andmask** will contain the value 1000000000000000, which is obtained by left-shifting 1,

fifteen places. If the variable **n**'s most significant bit is 0, then **k** would contain a value 0, otherwise it would contain a non-zero value. If **k** contains 0 then **printf()** will print out 0 otherwise it will print out 1.

On the second go-around of the loop, the value of **i** is decremented and hence the value of **andmask** changes, which will now be 0100000000000000. This checks whether the next most significant bit is 1 or 0, and prints it out accordingly. The same operation is repeated for all bits in the number.

Summary

- (a) To help manipulate hardware oriented data—individual bits rather than bytes a set of bitwise operators are used.
- (b) The bitwise operators include operators like one's complement, right-shift, left-shift, bitwise AND, OR, and XOR.
- (c) The one's complement converts all zeros in its operand to 1s and all 1s to 0s.
- (d) The right-shift and left-shift operators are useful in eliminating bits from a number—either from the left or from the right.
- (e) The bitwise AND operators is useful in testing whether a bit is on/off and in putting off a particular bit.
- (f) The bitwise OR operator is used to turn on a particular bit.
- (g) The XOR operator works almost same as the OR operator except one minor variation.

Exercise

[A] Answer the following:

- (a) The information about colors is to be stored in bits of a **char** variable called **color**. The bit number 0 to 6, each represent 7 colors of a rainbow, i.e. bit 0 represents violet, 1 represents

indigo, and so on. Write a program that asks the user to enter a number and based on this number it reports which colors in the rainbow does the number represents.

- (b) A company planning to launch a new newspaper in market conducts a survey. The various parameters considered in the survey were, the economic status (upper, middle, and lower class) the languages readers prefer (English, Hindi, Regional language) and category of paper (daily, supplement, tabloid). Write a program, which reads data of 10 respondents through keyboard, and stores the information in an array of integers. The bit-wise information to be stored in an integer is given below:

Bit Number	Information
0	Upper class
1	Middle class
2	Lower class
3	English
4	Hindi
5	Regional Language
6	Daily
7	Supplement
8	Tabloid

At the end give the statistical data for number of persons who read English daily, number of upper class people who read tabloid and number of regional language readers.

- (c) In an inter-college competition, various sports and games are played between different colleges like cricket, basketball, football, hockey, lawn tennis, table tennis, carom and chess. The information regarding the games won by a particular college is stored in bit numbers 0, 1, 2, 3, 4, 5, 6, 7 and 8 respectively of an integer variable called **game**. The college

that wins in 5 or more than 5 games is awarded the Champion of Champions trophy. If a number is entered through the keyboard, then write a program to find out whether the college won the Champion of the Champions trophy or not, along with the names of the games won by the college.

- (d) An animal could be either a canine (dog, wolf, fox, etc.), a feline (cat, lynx, jaguar, etc.), a cetacean (whale, narwhal, etc.) or a marsupial (koala, wombat, etc.). The information whether a particular animal is canine, feline, cetacean, or marsupial is stored in bit number 0, 1, 2 and 3 respectively of a integer variable called **type**. Bit number 4 of the variable **type** stores the information about whether the animal is Carnivore or Herbivore.

For the following animal, complete the program to determine whether the animal is a herbivore or a carnivore. Also determine whether the animal is a canine, feline, cetacean or a marsupial.

```
struct animal
{
    char name[30];
    int type;
}
struct animal a = { "OCELOT", 18 };
```

- (e) The time field in the directory entry is 2 bytes long. Distribution of different bits which account for hours, minutes and seconds is given below. Write a function which would receive the two-byte time entry and return to the calling function, the hours, minutes and seconds.

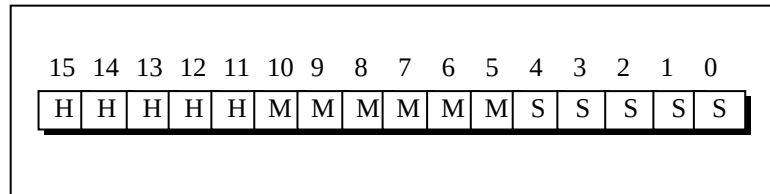


Figure 14.13

- (f) In order to save disk space information about student is stored in an integer variable. If bit number 0 is on then it indicates Ist year student, bit number 1 to 3 stores IInd year, IIIrd year and IVth year student respectively. The bit number 4 to 7 stores stream Mechanical, Chemical, Electronics and IT. Rest of the bits store room number. Based on the given data, write a program that asks for the room number and displays the information about the student, if its data exists in the array. The contents of array are,

```
int data[] = { 273, 548, 786, 1096 } ;
```

- (g) What will be the output of the following program:

```
main()
{
    int i = 32, j = 65, k, l, m, n, o, p ;
    k = i | 35 ; l = ~k ; m = i & j ;
    n = j ^ 32 ; o = j << 2 ; p = i >> 5 ;
    printf ( "\nk = %d l = %d m = %d", k, l, m ) ;
    printf ( "\nn = %d o = %d p = %d", n, o, p ) ;
}
```